don't have write access in your lab machine or want to flash your board in a one-off manner. Just run the command *docker run -it --rm --device=/dev/ttyACM0:/dev/ttyACM0 --entrypoint=ash esptool* and download the firmware in the container itself through the above shown *wget+curl* command. Now use *esptool erase-flash && esptool write-flash 0 /tmp/ESP32_GENERIC_C3_Latest.bin* to finish flashing your board.

Otherwise, execute the following command:

```
docker run --rm -v /tmp/
ESP32_GENERIC_C3_Latest.bin:/
tmp/ESP32_GENERIC_C3_Latest.
bin --device=/dev/ttyACM0:/
dev/ttyACM0 esptool
```

…to dump a help screen of the tool. Feel free to play with different esptool options and commands shown in the *Help* screen.

Finally, execute these commands to flash the board:

```
docker run --rm -v /tmp/
ESP32_GENERIC_C3_Latest.bin:/
tmp/ESP32_GENERIC_C3_Latest.
bin --device=/dev/ttyACM0:/
dev/ttyACM0 esptool erase-
flash

docker run --rm -v /tmp/
ESP32_GENERIC_C3_Latest.bin:/
tmp/ESP32_GENERIC_C3_Latest.
bin --device=/dev/ttyACM0:/
dev/ttyACM0 esptool write-
flash 0 /tmp/ESP32_GENERIC_C3_
Latest.bin
```

Congratulations! Now your Super Mini board is fully ready to serve you. Please note, the esptool *erase-flash* command is very handy to start with a clean slate in case your board breaks down later, especially after a firmware upgrade. Now your board has the MicroPython, so the pyboard and mpremote tools covered in the last article (*OSFY*, September 2026) are fully applicable to it.

Great! It's time to run some MicroPython code quickly, showing an onboard LED and inbuilt temperature sensor in action. Execute the command:

```
docker run --device=/
dev/ttyACM0:/dev/ttyACM0
pyboard  -c 'from machine
import Pin;from time
import sleep;p=Pin(8,
Pin.OUT, value=1);p.
toggle();sleep(3);p.toggle()'
```

…to flash the onboard blue LED. If you have noticed, the onboard LED is active low logic based, which means it is 'on' when Pin OUT is low and vice versa. This is opposite to the Pi Pico onboard LED logic.

Next, prepare the MicroPython code shown below, which executes on both the Super Mini board as well as the Pi Pico boards.

```
#--- Start of bleditemps.
py ---

from machine import Pin,
soft_reset, Signal
from os import uname
from time import sleep

pfrm = uname()[-1].split()
[-1].replace('-','')
if -1 == pfrm.find('ESP32C3'):
  ahgh = True
else:
  import esp32
  ahgh = False

pled = "LED"
if -1 != pfrm.find('ESP32C3'):
  # onboard LED is invert
logic based on ESP32C3 Super
Mini
  pled = 8

iled = Signal(Pin(pled, Pin.
OUT), invert=not(ahgh))

if -1 == pfrm.find('ESP32C3'):
  isnsr = ADC(4)
else:
  isnsr = None

tglf = True
while True:
  try:
    if tglf:
      iled.on()
    else:
      iled.off()

    if isnsr:
      conversion_factor = 3.3
/ 65535
      reading = isnsr.read_
u16() * conversion_factor
      # The temperature
sensor measures the Vbe
voltage of a biased bipolar
diode, connected to the fifth
ADC channel

      # Typically, Vbe =
0.706V at 27 degrees C,
with a slope of -1.721mV
(0.001721) per degree.
      itemp = 27 - (reading -
0.706)/0.001721
    else:
      itemp = esp32.mcu_
temperature()
    print(" itemp : %.2f C"
%itemp)

    sleep(1)
    tglf = not(tglf)
  except:
    iled.off()
    # due to reset the
ESP32C3 board got stuck in
running
    # after copying main.py
to it.
    if -1 == pfrm.
find('ESP32C3'):
      soft_reset()

#--- End of bleditemps.py ---
```

Now execute the command:

```
docker run --rm -v ${PWD}/
bleditemps.py:/etc/rpipico/
bleditemps.py:ro --device=/
dev/ttyACM0:/dev/ttyACM0
pyboard bleditemps.py
```

…to run the code on the Super Mini, flashing the onboard LED along with dumping the on-chip temperature sensor readings. Just press the *Reset* button on the board to end the current run and prepare it to run the next test code snippets. Like the Pi Pico boards, if you copy your code to a *main.py* file on the board then the Super Mini starts executing this code with every power on. Please note, our code uses a Signal class; this is to abstract Pin objects to take care of active high/low logic to keep the same on/off sequence across different devices. You'll see us making use of Signal almost everywhere going forward. All this should be enough to warm you up and feel confident to interface with the various useful sensors covered in the next section.